Type 3: Software Project

Adaptive Acoustic Auth (AAA) Engine

A Plug-and-Play Biometric Security Middleware with Deepfake Immunity

23CSE498 – Project Phase 2 (Panel Review 1) | **Team Number: 90**

S.No	Register Number	Student Name
1	CB.SC.U4CSE23431	M Nandan Reddy
2	CB.SC.U4CSE23434	M Navadeep Kumar
3	CB.SC.U4CSE23464	Parise Aditya
4	CB.SC.U4CSE23467	AVKB Surya
5	CB.SC.U4CSE23534	M Karthik Srijan

Guide: **Dr. Harini N**, Associate Professor, Department of CSE

September 18, 2026

Presentation Outline

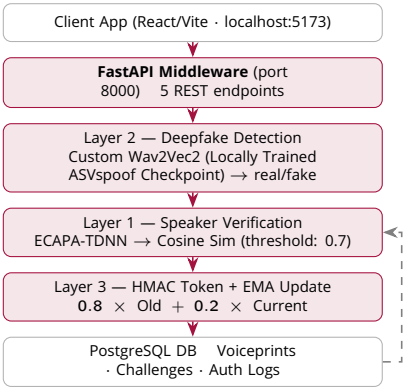
1. Architecture & Workflow Design
2. Implementation Progress
3. Technical Knowledge
4. Technical Enrichment
5. Plans for Upcoming Semesters
6. References

1. Application Architecture & Workflow Design

Rubric: Workflow, Module Interaction, UI Flow, DB Schema, APIs & Deployment (5 Marks)

System Architecture Components:

- **End-to-End Workflow:** React (Vite) Client → FastAPI Middleware (port 8000) → L2 → L1 → L3 → PostgreSQL
- **Module Interactions:** `layer1_speaker.py` (Surya), `layer2_deepfake.py` (Nandan), `layer3_adaptive.py` (Aditya) — orchestrated by `main.py` (Navadeep)
- **Data Tier:** PostgreSQL with `users`, `voiceprints` (ARRAY(Float)), `challenges`, `auth_logs`
- **Tech Stack:** React/Vite frontend · FastAPI + SQLAlchemy backend · PyTorch / HuggingFace / SpeechBrain AI core · Docker Compose
- **CORS:** `localhost:5173` & `localhost:3000`



1b. Database Schema & API Contract

Shared Contract — Source of Truth for All 4 Modules

PostgreSQL Schema (4 Tables via SQLAlchemy ORM):

- `users` — UUID (PK), name, email (unique), created_at
- `voiceprints` — user_id FK, `embedding ARRAY(Float)`, sample_count INT, updated_at
- `challenges` — user_id FK, phrase, token TEXT, expires_at, used BOOLEAN
- `auth_logs` — user_id FK, challenge_id FK, l2_label, l2_confidence, l1_score, result, layer_blocked, created_at

Audio Spec:

WAV · 16 kHz · Mono Enroll: min 5s Verify: min 2–3s

Browser audio auto-converted via `audioEncoder.js` (WebM → 16kHz WAV)

5-Endpoint REST API:

Endpoint	Method	Returns
`/auth/register`	POST	`{user_id}`
`/auth/enroll`	POST	`{status, sample_count}`
`/auth/challenge`	GET	phrase + token
`/auth/verify`	POST	full result JSON
`/users/{id}/voiceprint-history`	GET	history list

Verify Response Shape:

`{result, confidence, layer_blocked, session_id, reason, l2_label, l2_confidence, l1_score}`

Decision Thresholds (`main.py`):

L2 spoof threshold: `0.5` L1 similarity threshold: `0.7`

2. Implementation Progress & Module Functionality

Rubric: Major Modules Implemented, Integrated & Demonstrated (8 Marks)

Phase 0 & Phase 1 — Fully Completed Across All 4 Tracks:

Module / File	Implementation State	Status
Infra + Backend `main.py, db.py, models.py`	PostgreSQL via SQLAlchemy ORM; all 5 FastAPI routes live; Pydantic v2 response models; L2→L1→L3 pipeline in `POST /auth/verify`; EMA applied on `/enroll` AND on successful `/verify`	**Integrated**
Layer 1 Speaker `layer1_speaker.py`	Custom ECAPA-TDNN instantiated from local .pth weights; `LocalStrategy.COPY` bypasses Windows symlink restriction; librosa fallback for browser audio; 192-dim cosine sim normalised to 0–1	**Functional**
Layer 2 Deepfake `layer2_deepfake.py`	Locally fine-tuned Wav2Vec2 checkpoint (trained on ASVspoof 2019 LA); lazy-loaded from disk on first request; maps {0:real, 1:fake} logits → softmax → {bonafide/spoof, confidence}; CUDA auto-detected	**Functional**

Layer 3 + Frontend
`layer3_adaptive.py`
`/frontend`
	HMAC-SHA256 tokens (`nonce.expires_ts.sig`, 90s TTL, single-use, timing-safe `compare_digest` validation) enforce production-grade replay protection; dynamic challenge phrases (60% randomised *Say: {num} {colour} {object} {num}* + 40% static) block pre-recorded injection; React Vite frontend with 5 pages	**Integrated**

Phase 2 Integration: All 4 branches merged; enroll → challenge → verify end-to-end tested — ACCEPT + REJECT (Layer 2 blocked & Layer 1 blocked) both demonstrated live.

2b. Layer API Signatures — Frozen Interface Definitions

Fixed function signatures each module must expose — `main.py` imports by name, no deviations allowed

Layer 1 — `layer1_speaker.py`:

```
def enroll_speaker(
  audio_bytes:  bytes) -> dict:
    # 16kHz WAV -> ECAPA-TDNN
    # {“embedding”:  list[float]} # 192-dim

def verify_speaker(
  audio_bytes:  bytes,
  stored_embedding:  list[float]
) -> dict:
    # cosine sim normalised 0-1
    # {“score”:  float,
    # “embedding”:  list[float]}
```

Layer 2 — `layer2_deepfake.py`:

```
def detect_spoof(
  audio_bytes:  bytes) -> dict:
    # loads local weights:  l2_wav2vec2_best.pth
    # lazy-loaded; CUDA auto-detected
    # {“label”:  “bonafide”|“spoof”,
    # “confidence”:  float}
```

Layer 3 — `layer3_adaptive.py`:

```
def generate_challenge(
  user_id:  str) -> dict:
    # HMAC-SHA256, 90s TTL
    # token = nonce.expires_ts.sig
    # {phrase, token, expires_at}

def verify_token(
  token:  str) -> bool:
    # single-use + TTL + compare_digest

def update_voiceprint(
  old_emb:  list[float],
  new_emb:  list[float],
  alpha:  float = 0.2
) -> list[float]:
    # 0.8*Old + 0.2*Current
```

Deployed Local Baselines:

L1: ECAPA-TDNN (Local Checkpoint, CPU)
L2: Wav2Vec2 (ASVspoof 2019 Local Checkpoint, CUDA auto)

3. Technical Knowledge & Architectural Decisions

Rubric: In-depth Understanding of Architecture, Implementation & Tech Stack (5 Marks)

Tech Stack Justification:

- **FastAPI** — async multipart upload; `Depends(get_db)` for per-request DB sessions; Pydantic v2 response models; stateless — pluggable over any existing voice system
- **ECAPA-TDNN** — channel-attention + attentive pooling; 192-dim embeddings; `LocalStrategy.COPY` bypasses Windows symlink restriction; librosa fallback for WebM audio
- **Custom Wav2Vec2 (Local ASVspoof Checkpoint)** — fine-tuned in-house on real/fake pairs; {0:real, 1:fake} logits → softmax; lazy-loaded (no cold-start on enroll)
- **SQLAlchemy ORM** — `ARRAY(Float)` native PG column; no external vector DB needed
- **Docker Compose** — backend + frontend services; `uvicorn -reload`

Design Patterns & Security:

- **Layered Gate:** L2 (spoof gate) → L1 (identity gate) → L3 (token gate) — TTS attacks rejected before embedding compute
- **HMAC-SHA256 Tokens:** format `nonce.expires_ts.sig`; 90s TTL; single-use in-memory store; `compare_digest` timing-safe check — production-grade defense against replay and token-guessing attacks
- **EMA Adaptation ($\alpha = 0.2$):** applied on `/enroll` AND on each successful `/verify` — voiceprint adapts over time
- **Dynamic Challenge:** 60% randomised phrase (*Say: {num} {colour} {object} {num}*), 40% static — prevents pre-recorded injection and enforces liveness at the protocol level

Scalability Target: p95 latency $< 1.5\text{s} \cdot 50$ concurrent users (Locust — Phase 3)

4. Technical Enrichment Activity Choices

Professional Online Certification Details (Individual Mandatory: 10–20 Hours)

- **Individual MOOC / Online Certification Selection:**

S.No	Student Name	Chosen MOOC / Course Title	Platform	Hours
1	M Nandan Reddy	Mastering Generative Voice AI: From Tokens to Agentic TTS	Udemy	18 Hrs
2	M Navadeep Kumar	Mastering Speech & Language Models: From ASR to Emotion AI	Udemy	18 Hrs
3	Parise Aditya	REST APIs with Python, Flask, Docker, AWS, Git & SQLAlchemy	Udemy	16 Hrs
4	AVKB Surya	HuggingFace Audio Course — Audio Classification & Transformers	HuggingFace	15 Hrs
5	M Karthik Srijan	Complete Cryptography Guide: Secure Data & Networks	Udemy	14 Hrs

5. Plans for Upcoming Semesters

Current State: Baseline models (ECAPA-TDNN L1, Wav2Vec2 L2 — both locally trained) functional & integrated.

Middleware-First Redesign:

- Package AAA as a drop-in, framework-agnostic **auth middleware** (pip-installable), not a standalone app
- Ready-made adapter wrappers for **FastAPI, Flask, Express.js** — one-line plug-in over any existing voice system
- Config-driven thresholds (L1/L2) & pluggable model backends — swap models without touching call sites

Layer 1 Upgrade — Speaker Verification:

- Benchmark ECAPA-TDNN EER on VoxCeleb1 — target $\leq$ **0.87%** (vs 3.1% x-vector baseline)
- Tune cosine threshold beyond current 0.7 on dev split to minimise FAR/FRR

Layer 2 Upgrade — Deepfake Detection:

- Upgrade our existing locally trained Wav2Vec2 baseline to fine-tuned **Wav2Vec2BERT** on ASVspoof 2019 LA (Amrita HPC, V100, FP16)
- Evaluate on ASVspoof 2021 DF — beat LFCC-LCNN ($\sim$15–20% EER) & RawNet2 ($\sim$10–14% EER)
- Config-only checkpoint swap — zero API code change

In-House Deepfake Voice Generator:

- Build our own **voice-cloning TTS generator** (YourTTS/XTTS zero-shot, fine-tuned) to synthesise adversarial spoof samples in-house
- Closes-the-loop red-teaming: generated deepfakes fed back into Layer 2 training/eval instead of relying only on public spoof datasets
- Target: $>$ **95%** Layer 2 detection rate against our own generated attacks

Challenge-Response Enhancement:

- **Whisper-tiny** ASR — confirm spoken words match challenge phrase (closes semantic liveness gap)
- Migrate token store from in-memory dict to **Redis** (multi-worker safe)

Voiceprint & Drift:

- `drift_history` table — log old/new embedding, cosine delta, timestamp
- Enroll min: 5s $\rightarrow$ 10s; EMA $\alpha \in \{0.7, 0.8, 0.9\}$ tuning on real drift data

Security & Evaluation:

- Rate limiting on `/auth/verify`; IP abuse detection in `auth_logs`
- Locust: 50 concurrent users, p95 $<$ **1.5**s

References I

- J. Yamagishi et al., “ASVspoof 2021: Accelerating Progress in Spoofed and Deepfake Speech Detection,” 2021.
- K. Liou et al., “Enhancing ECAPA-TDNN with Feature Processing Module and Attention Mechanism for Speaker Verification,” 2024.
- A. Borodin et al., “AASIST3: KAN-Enhanced AASIST Speech Deepfake Detection using SSL Features and Additional Regularization for the ASVspoof 2024 Challenge,” 2024.
- “Robust Audio Deepfake Detection: Exploring Front-/Back-End Combinations and Data Augmentation Strategies for the ASVspoof5 Challenge,” 2024.
- S. Wani et al., “HCN-TA: Hierarchical Capsule Network with Temporal Attention for a Generalizable Approach to Audio Deepfake Detection,” 2025.
- Ramnadh et al., “Audio-Based Deepfake Detection Leveraging ECAPA-TDNN and TIMIT-TTS Corpus,” 2025.

Thank You

Questions & Discussion

Adaptive Acoustic Auth (AAA) Engine

A Plug-and-Play Deepfake-Immune Biometric Security Middleware

Novelty 1
Dynamic Challenge-Response + Deepfake Detection Pipeline

Novelty 2
Adaptive Voiceprint Update (EMA, $\alpha = 0.2$)

Novelty 3
Plug-and-Play FastAPI Middleware